

- **read\_verilog dadda4bit.v:**  
The tool reads the Verilog source code, parses the syntax, and translates it into an Abstract Syntax Tree (AST).  
It then generates Yosys's internal representation, known as RTLIL (Register-Transfer-Level Intermediate Language).
- **hierarchy -check -top dadda4:**  
This command analyzes the module dependencies to build the design hierarchy.  
The -check flag ensures there are no missing modules  
The -top dadda4 explicitly designates the 4-bit multiplier as the top-level module.
- **proc:**  
This converts high-level Verilog procedural blocks (like always or initial blocks) into hardware primitives like multiplexers, latches, and flip-flops.  
Because the Dadda multiplier relies on continuous assign statements , this step mostly performs basic clean-up and removes empty switches .
- **opt:**  
This is an optimization pass. It simplifies the logic by performing constant folding (like resolving 1 & 0 to 0), removing unused wires, and eliminating dead code.
- **techmap:**  
This translates the RTLIL representation into technology-independent generic logic gates.
- **synth -top dadda4:**  
This is actually a powerful "macro" command. While it redundantly re-runs hierarchy, proc, and opt, its main purpose here is to execute the abc pass.  
This invokes the UC Berkeley ABC tool, which takes the mapped logic and performs heavy boolean optimization, transforming the design into the final minimal set of logic gates (AND, OR, XOR)
- **write\_verilog dadda4synth.v:**  
This exports the synthesized, gate-level netlist into a new Verilog file so that it can be passed into the Atalanta ATPG tool .

---

```
yosys> read_verilog dadda4bit.v
```

1. Executing Verilog-2005 frontend: dadda4bit.v

Parsing Verilog input from `dadda4bit.v' to **AST representation**.

(An Abstract Syntax Tree (AST) representation of Verilog code is a tree-based data structure that parses source code into a hierarchical, structured format, stripping away formatting and punctuation to focus on syntactic structure. It represents modules, assignments, and expressions as nodes, enabling tools like compilers, linters, and simulators to analyze code structure, verify, and transform it)

Generating **RTLIL representation** for module `dadda4'.

(RTLIL (Register-Transfer-Level Intermediate Language) is the main internal data structure used by the Yosys synthesis tool to represent a digital design. It is a hardware-oriented, technology-agnostic format that bridges the gap between high-level Verilog code and a final, optimized gate-level netlist.)

Successfully finished Verilog frontend.

```
yosys> hierarchy -check -top dadda4
```

2. Executing HIERARCHY pass (managing design hierarchy).

2.1. Analyzing design hierarchy..

Top module: \dadda4

2.2. Analyzing design hierarchy..

Top module: \dadda4

Removed 0 unused modules.

(what happens in this step?)

This step resolves the structural hierarchy of your design. The tool starts at the specified top-level module (in this case, dadda4 ) and recursively identifies any submodules instantiated inside it. The -check flag acts as a sanity check to ensure no referenced modules are missing. Finally, it strips away any modules that were parsed but are not actually used in the active hierarchy.

yosys> proc

### 3. Executing PROC pass (convert processes to netlists).

(what happens in proc, what are the different “PROC\_<whatever>” mentioned below?, where <whatever> = {CLEAN, RMDEAD, PRUNE, INIT, ARST, ROM, MUX, DLATCH, DFF, MEMWR, EXPR})

The proc pass translates Verilog procedural blocks (like always, initial, and task blocks) into actual hardware structures. Because the 4-bit Dadda multiplier relies on continuous assign statements rather than procedural blocks, proc does not have much to do here, but here is what the sub-passes do in a typical design:

- PROC\_CLEAN: Removes empty switch (case/if) statements.
- PROC\_RMDEAD: Detects and eliminates dead or unreachable code branches within decision trees.
- PROC\_PRUNE: Removes redundant assignments (like a variable being assigned the same value multiple times).
- PROC\_INIT: Extracts initialization attributes to set power-on states for registers.
- PROC\_ARST: Detects asynchronous reset patterns in always blocks to infer the correct type of flip-flop.
- PROC\_ROM: Converts large, dense case statements or arrays into Read-Only Memory (ROM) structures.
- PROC\_MUX: Converts remaining decision trees (if/else) into multiplexers.
- PROC\_DLATCH: Infers transparent D-latches for variables assigned in some, but not all, branches of a combinational block (often an unintended consequence of incomplete if statements).
- PROC\_DFF: Infers edge-triggered D-flip-flops for sequential always @(posedge clk) blocks.
- PROC\_MEMWR: Converts array writes inside processes into specific memory-write ports.
- PROC\_EXPR: Performs basic expression optimization before the next stages.

- 3.1. Executing PROC\_CLEAN pass (remove empty switches from decision trees).  
Cleaned up 0 empty switches.
- 3.2. Executing PROC\_RMDEAD pass (remove dead branches from decision trees).  
Removed a total of 0 dead cases.
- 3.3. Executing PROC\_PRUNE pass (remove redundant assignments in processes).  
Removed 0 redundant assignments.  
Promoted 0 assignments to connections.
- 3.4. Executing PROC\_INIT pass (extract init attributes).
- 3.5. Executing PROC\_ARST pass (detect async resets in processes).
- 3.6. Executing PROC\_ROM pass (convert switches to ROMs).  
Converted 0 switches.
- 3.7. Executing PROC\_MUX pass (convert decision trees to multiplexers).
- 3.8. Executing PROC\_DLATCH pass (convert process syncs to latches).
- 3.9. Executing PROC\_DFF pass (convert process syncs to FFs).
- 3.10. Executing PROC\_MEMWR pass (convert process memory writes to cells).
- 3.11. Executing PROC\_CLEAN pass (remove empty switches from decision trees).  
Cleaned up 0 empty switches.
- 3.12. Executing OPT\_EXPR pass (perform const folding).  
Optimizing module dadda4.

yosys> opt

#### 4. Executing OPT pass (performing simple optimizations).

(what is opt about? what are the "simple optimizations"?)

(what are the "const foldings"?)

(what are the different "OPT\_<whatever>" mentioned below?, where <whatever> = {MERGE, MUXTREE, REDUCE, DFF, EXPR, CLEAN})

The opt command runs a series of optimization routines to shrink the design's area and improve timing before technology mapping.

Constant Folding (OPT\_EXPR): If the inputs to a logic gate are known, fixed constants, the tool pre-calculates the output at compile time. For example, if a logic branch simplifies to  $A \& 1'b0$ , the tool "folds" that entire expression into a constant 0, removing the need for an AND gate.

- OPT\_MERGE: Identifies and merges identical logic cells that perform the exact same function on the exact same inputs, avoiding redundant hardware.
- OPT\_MUXTREE: Analyzes multiplexer trees and removes dead branches (e.g., if a MUX select line is permanently tied to 1, the 0 input logic is deleted).
- OPT\_REDUCE: Simplifies wide reduction operators (like a 16-input AND) and consolidates inputs to MUXes.
- OPT\_DFF: Optimizes flip-flops (e.g., removing flip-flops whose outputs are never read, or merging identical flip-flops).
- OPT\_CLEAN: A garbage collection pass. It sweeps through and deletes completely unused cells and dangling wires .

##### 4.1. Executing OPT\_EXPR pass (perform const folding).

Optimizing module dadda4.

##### 4.2. Executing OPT\_MERGE pass (detect identical cells).

Finding identical cells in module `dadda4'.

Removed a total of 0 cells.

##### 4.3. Executing OPT\_MUXTREE pass (detect dead branches in mux trees).

Running muxtree optimizer on module dadda4..

Creating internal representation of mux trees.

No muxes found in this module.

Removed 0 multiplexer ports.

##### 4.4. Executing OPT\_REDUCE pass (consolidate \$\*mux and \$reduce\_\* inputs).

Optimizing cells in module dadda4.

Performed a total of 0 changes.

##### 4.5. Executing OPT\_MERGE pass (detect identical cells).

Finding identical cells in module `dadda4'.

Removed a total of 0 cells.

4.6. Executing OPT\_DFF pass (perform DFF optimizations).

4.7. Executing OPT\_CLEAN pass (remove unused cells and wires).

Finding unused cells or wires in module \dadda4..

Removed 0 unused cells and 40 unused wires.

<suppressed ~1 debug messages>

4.8. Executing OPT\_EXPR pass (perform const folding).

Optimizing module dadda4.

4.9. Rerunning OPT passes. (Maybe there is more to do..)

4.10. Executing OPT\_MUXTREE pass (detect dead branches in mux trees).

Running muxtree optimizer on module \dadda4..

Creating internal representation of mux trees.

No muxes found in this module.

Removed 0 multiplexer ports.

4.11. Executing OPT\_REDUCE pass (consolidate \$\*mux and \$reduce\_\* inputs).

Optimizing cells in module \dadda4.

Performed a total of 0 changes.

4.12. Executing OPT\_MERGE pass (detect identical cells).

Finding identical cells in module '\dadda4'.

Removed a total of 0 cells.

4.13. Executing OPT\_DFF pass (perform DFF optimizations).

4.14. Executing OPT\_CLEAN pass (remove unused cells and wires).

Finding unused cells or wires in module \dadda4..

4.15. Executing OPT\_EXPR pass (perform const folding).

Optimizing module dadda4.

4.16. Finished OPT passes. (There is nothing left to do.)

yosys> techmap

## 5. Executing TECHMAP pass (map to technology primitives).

### 5.1. Executing Verilog-2005 frontend: /usr/bin/./share/yosys/techmap.v

Parsing Verilog input from `/usr/bin/./share/yosys/techmap.v' to AST representation.

Generating RTLIL representation for module `\_90\_simplemap\_bool\_ops'.

Generating RTLIL representation for module `\_90\_simplemap\_reduce\_ops'.

Generating RTLIL representation for module `\_90\_simplemap\_logic\_ops'.

Generating RTLIL representation for module `\_90\_simplemap\_compare\_ops'.

Generating RTLIL representation for module `\_90\_simplemap\_various'.

Generating RTLIL representation for module `\_90\_simplemap\_registers'.

Generating RTLIL representation for module `\_90\_shift\_ops\_shr\_shl\_sshl\_sshr'.

Generating RTLIL representation for module `\_90\_shift\_shiftx'.

Generating RTLIL representation for module `\_90\_fa'.

Generating RTLIL representation for module `\_90\_lcu'.

Generating RTLIL representation for module `\_90\_alu'.

Generating RTLIL representation for module `\_90\_macc'.

Generating RTLIL representation for module `\_90\_alumacc'.

Generating RTLIL representation for module `\$\_div\_mod\_u'.

Generating RTLIL representation for module `\$\_div\_mod\_trunc'.

Generating RTLIL representation for module `\_90\_div'.

Generating RTLIL representation for module `\_90\_mod'.

Generating RTLIL representation for module `\$\_div\_mod\_floor'.

Generating RTLIL representation for module `\_90\_divfloor'.

Generating RTLIL representation for module `\_90\_modfloor'.

Generating RTLIL representation for module `\_90\_pow'.

Generating RTLIL representation for module `\_90\_pmux'.

Generating RTLIL representation for module `\_90\_demux'.

Generating RTLIL representation for module `\_90\_lut'.

Successfully finished Verilog frontend.

### 5.2. Continuing TECHMAP pass.

Using extmapper simplemap for cells of type \$and.

Using extmapper simplemap for cells of type \$xor.

Using extmapper simplemap for cells of type \$or.

No more expansions possible.

<suppressed ~153 debug messages>



yosys> synth -top dadda4

6. Executing SYNTH pass.

6.1. Executing HIERARCHY pass (managing design hierarchy).

6.1.1. Analyzing design hierarchy..

Top module: \dadda4

6.1.2. Analyzing design hierarchy..

Top module: \dadda4

Removed 0 unused modules.

6.2. Executing PROC pass (convert processes to netlists).

6.2.1. Executing PROC\_CLEAN pass (remove empty switches from decision trees).

Cleaned up 0 empty switches.

6.2.2. Executing PROC\_RMDEAD pass (remove dead branches from decision trees).

Removed a total of 0 dead cases.

6.2.3. Executing PROC\_PRUNE pass (remove redundant assignments in processes).

Removed 0 redundant assignments.

Promoted 0 assignments to connections.

6.2.4. Executing PROC\_INIT pass (extract init attributes).

6.2.5. Executing PROC\_ARST pass (detect async resets in processes).

6.2.6. Executing PROC\_ROM pass (convert switches to ROMs).

Converted 0 switches.

6.2.7. Executing PROC\_MUX pass (convert decision trees to multiplexers).

6.2.8. Executing PROC\_DLATCH pass (convert process syncs to latches).

6.2.9. Executing PROC\_DFF pass (convert process syncs to FFs).

6.2.10. Executing PROC\_MEMWR pass (convert process memory writes to cells).

6.2.11. Executing PROC\_CLEAN pass (remove empty switches from decision trees).

Cleaned up 0 empty switches.

6.2.12. Executing OPT\_EXPR pass (perform const folding).

Optimizing module dadda4.

6.3. Executing OPT\_EXPR pass (perform const folding).

Optimizing module dadda4.

6.4. Executing OPT\_CLEAN pass (remove unused cells and wires).  
Finding unused cells or wires in module \dadda4..

6.5. Executing CHECK pass (checking for obvious problems).  
Checking module dadda4...  
Found and reported 0 problems.

6.6. Executing OPT pass (performing simple optimizations).

6.6.1. Executing OPT\_EXPR pass (perform const folding).  
Optimizing module dadda4.

6.6.2. Executing OPT\_MERGE pass (detect identical cells).  
Finding identical cells in module '\dadda4'.  
Removed a total of 0 cells.

6.6.3. Executing OPT\_MUXTREE pass (detect dead branches in mux trees).  
Running muxtree optimizer on module \dadda4..  
Creating internal representation of mux trees.  
No muxes found in this module.  
Removed 0 multiplexer ports.

6.6.4. Executing OPT\_REDUCE pass (consolidate \$\*mux and \$reduce\_\* inputs).  
Optimizing cells in module \dadda4.  
Performed a total of 0 changes.

6.6.5. Executing OPT\_MERGE pass (detect identical cells).  
Finding identical cells in module '\dadda4'.  
Removed a total of 0 cells.

6.6.6. Executing OPT\_DFF pass (perform DFF optimizations).

6.6.7. Executing OPT\_CLEAN pass (remove unused cells and wires).  
Finding unused cells or wires in module \dadda4..

6.6.8. Executing OPT\_EXPR pass (perform const folding).  
Optimizing module dadda4.

6.6.9. Finished OPT passes. (There is nothing left to do.)

(What are the FSM passes?)

(what are the different “FSM\_<whatever>” mentioned below?, where <whatever> = {DETECT, EXTRACT, OPT, CLEAN, RECODE, INFO, MAP, EXPR})

Finite State Machine (FSM) optimization attempts to find state machines written in behavioral Verilog and re-map them for better hardware efficiency. Because the Dadda multiplier is a pure combinational circuit without a clock, these passes find nothing. For sequential designs, they do the following:

- FSM\_DETECT: Looks for patterns of flip-flops and logic that resemble a state register.
- FSM\_EXTRACT: Pulls that specific logic out and replaces it with a dedicated internal \$fsm cell.
- FSM\_OPT: Performs boolean optimization specifically on the state transitions (like removing unreachable states).
- FSM\_RECODE: Changes the state encoding (like switching from Binary encoding to One-Hot encoding) to optimize for speed or area.
- FSM\_INFO: Dumps information about the detected state machines into the log.
- FSM\_MAP: Translates the optimized \$fsm cell back into discrete logic gates and flip-flops.

6.7. Executing **FSM pass** (extract and optimize FSM).

6.7.1. Executing FSM\_DETECT pass (finding FSMs in design).

6.7.2. Executing FSM\_EXTRACT pass (extracting FSM from design).

6.7.3. Executing FSM\_OPT pass (simple optimizations of FSMs).

6.7.4. Executing OPT\_CLEAN pass (remove unused cells and wires).  
Finding unused cells or wires in module \dadda4..

6.7.5. Executing FSM\_OPT pass (simple optimizations of FSMs).

6.7.6. Executing FSM\_RECODE pass (re-assigning FSM state encoding).

6.7.7. Executing FSM\_INFO pass (dumping all available information on FSM cells).

6.7.8. Executing FSM\_MAP pass (mapping FSMs to basic logic).

6.8. Executing OPT pass (performing simple optimizations).

6.8.1. Executing OPT\_EXPR pass (perform const folding).  
Optimizing module dadda4.

6.8.2. Executing OPT\_MERGE pass (detect identical cells).

Finding identical cells in module `dadda4'.  
Removed a total of 0 cells.

6.8.3. Executing OPT\_MUXTREE pass (detect dead branches in mux trees).  
Running muxtree optimizer on module dadda4..  
Creating internal representation of mux trees.  
No muxes found in this module.  
Removed 0 multiplexer ports.

6.8.4. Executing OPT\_REDUCE pass (consolidate \$\*mux and \$reduce\_\* inputs).  
Optimizing cells in module dadda4.  
Performed a total of 0 changes.

6.8.5. Executing OPT\_MERGE pass (detect identical cells).  
Finding identical cells in module `dadda4'.  
Removed a total of 0 cells.

6.8.6. Executing OPT\_DFF pass (perform DFF optimizations).

6.8.7. Executing OPT\_CLEAN pass (remove unused cells and wires).  
Finding unused cells or wires in module dadda4..

6.8.8. Executing OPT\_EXPR pass (perform const folding).  
Optimizing module dadda4.

6.8.9. Finished OPT passes. (There is nothing left to do.)

6.9. Executing WREDUCE pass (reducing word size of cells).

6.10. Executing PEEPOPT pass (run peephole optimizers).

6.11. Executing OPT\_CLEAN pass (remove unused cells and wires).  
Finding unused cells or wires in module dadda4..

6.12. Executing ALUMACC pass (create \$alu and \$macc cells).  
Extracting \$alu and \$macc cells in module dadda4:  
created 0 \$alu and 0 \$macc cells.

6.13. Executing SHARE pass (SAT-based resource sharing).

6.14. Executing OPT pass (performing simple optimizations).

6.14.1. Executing OPT\_EXPR pass (perform const folding).  
Optimizing module dadda4.

6.14.2. Executing OPT\_MERGE pass (detect identical cells).  
Finding identical cells in module `dadda4'.  
Removed a total of 0 cells.

6.14.3. Executing OPT\_MUXTREE pass (detect dead branches in mux trees).

Running muxtree optimizer on module \dadda4..

Creating internal representation of mux trees.

No muxes found in this module.

Removed 0 multiplexer ports.

6.14.4. Executing OPT\_REDUCE pass (consolidate \$\*mux and \$reduce\_\* inputs).

Optimizing cells in module \dadda4.

Performed a total of 0 changes.

6.14.5. Executing OPT\_MERGE pass (detect identical cells).

Finding identical cells in module '\dadda4'.

Removed a total of 0 cells.

6.14.6. Executing OPT\_DFF pass (perform DFF optimizations).

6.14.7. Executing OPT\_CLEAN pass (remove unused cells and wires).

Finding unused cells or wires in module \dadda4..

6.14.8. Executing OPT\_EXPR pass (perform const folding).

Optimizing module dadda4.

6.14.9. Finished OPT passes. (There is nothing left to do.)

6.15. Executing MEMORY pass.

6.15.1. Executing OPT\_MEM pass (optimize memories).

Performed a total of 0 transformations.

6.15.2. Executing OPT\_MEM\_PRIORITY pass (removing unnecessary memory write priority relations).

Performed a total of 0 transformations.

6.15.3. Executing OPT\_MEM\_FEEDBACK pass (finding memory read-to-write feedback paths).

6.15.4. Executing MEMORY\_BMUX2ROM pass (converting muxes to ROMs).

6.15.5. Executing MEMORY\_DFF pass (merging \$dff cells to \$memrd).

6.15.6. Executing OPT\_CLEAN pass (remove unused cells and wires).

Finding unused cells or wires in module \dadda4..

6.15.7. Executing MEMORY\_SHARE pass (consolidating \$memrd/\$memwr cells).

6.15.8. Executing OPT\_MEM\_WIDEN pass (optimize memories where all ports are wide).

Performed a total of 0 transformations.

6.15.9. Executing OPT\_CLEAN pass (remove unused cells and wires).  
Finding unused cells or wires in module \dadda4..

6.15.10. Executing MEMORY\_COLLECT pass (generating \$mem cells).

6.16. Executing OPT\_CLEAN pass (remove unused cells and wires).  
Finding unused cells or wires in module \dadda4..

6.17. Executing OPT pass (performing simple optimizations).

6.17.1. Executing OPT\_EXPR pass (perform const folding).  
Optimizing module dadda4.

6.17.2. Executing OPT\_MERGE pass (detect identical cells).  
Finding identical cells in module `dadda4'.  
Removed a total of 0 cells.

6.17.3. Executing OPT\_DFF pass (perform DFF optimizations).

6.17.4. Executing OPT\_CLEAN pass (remove unused cells and wires).  
Finding unused cells or wires in module \dadda4..

6.17.5. Finished fast OPT passes.

6.18. Executing MEMORY\_MAP pass (converting memories to logic and flip-flops).

6.19. Executing OPT pass (performing simple optimizations).

6.19.1. Executing OPT\_EXPR pass (perform const folding).  
Optimizing module dadda4.

6.19.2. Executing OPT\_MERGE pass (detect identical cells).  
Finding identical cells in module `dadda4'.  
Removed a total of 0 cells.

6.19.3. Executing OPT\_MUXTREE pass (detect dead branches in mux trees).  
Running muxtree optimizer on module \dadda4..  
Creating internal representation of mux trees.  
No muxes found in this module.  
Removed 0 multiplexer ports.

6.19.4. Executing OPT\_REDUCE pass (consolidate \$\*mux and \$reduce\_\* inputs).  
Optimizing cells in module \dadda4.  
Performed a total of 0 changes.

6.19.5. Executing OPT\_MERGE pass (detect identical cells).  
Finding identical cells in module `dadda4'.

Removed a total of 0 cells.

6.19.6. Executing OPT\_SHARE pass.

6.19.7. Executing OPT\_DFF pass (perform DFF optimizations).

6.19.8. Executing OPT\_CLEAN pass (remove unused cells and wires).  
Finding unused cells or wires in module \dadda4..

6.19.9. Executing OPT\_EXPR pass (perform const folding).  
Optimizing module dadda4.

6.19.10. Finished OPT passes. (There is nothing left to do.)

6.20. Executing TECHMAP pass (map to technology primitives).

6.20.1. Executing Verilog-2005 frontend: /usr/bin/./share/yosys/techmap.v  
Parsing Verilog input from `/usr/bin/./share/yosys/techmap.v' to AST representation.  
Generating RTLIL representation for module `\_90\_simplemap\_bool\_ops'.  
Generating RTLIL representation for module `\_90\_simplemap\_reduce\_ops'.  
Generating RTLIL representation for module `\_90\_simplemap\_logic\_ops'.  
Generating RTLIL representation for module `\_90\_simplemap\_compare\_ops'.  
Generating RTLIL representation for module `\_90\_simplemap\_various'.  
Generating RTLIL representation for module `\_90\_simplemap\_registers'.  
Generating RTLIL representation for module `\_90\_shift\_ops\_shr\_shl\_sshl\_sshr'.  
Generating RTLIL representation for module `\_90\_shift\_shiftx'.  
Generating RTLIL representation for module `\_90\_fa'.  
Generating RTLIL representation for module `\_90\_lcu'.  
Generating RTLIL representation for module `\_90\_alu'.  
Generating RTLIL representation for module `\_90\_macc'.  
Generating RTLIL representation for module `\_90\_alumacc'.  
Generating RTLIL representation for module `\$\_div\_mod\_u'.  
Generating RTLIL representation for module `\$\_div\_mod\_trunc'.  
Generating RTLIL representation for module `\_90\_div'.  
Generating RTLIL representation for module `\_90\_mod'.  
Generating RTLIL representation for module `\$\_div\_mod\_floor'.  
Generating RTLIL representation for module `\_90\_divfloor'.  
Generating RTLIL representation for module `\_90\_modfloor'.  
Generating RTLIL representation for module `\_90\_pow'.  
Generating RTLIL representation for module `\_90\_pmux'.  
Generating RTLIL representation for module `\_90\_demux'.  
Generating RTLIL representation for module `\_90\_lut'.  
Successfully finished Verilog frontend.

6.20.2. Continuing TECHMAP pass.

No more expansions possible.

<suppressed ~73 debug messages>

6.21. Executing OPT pass (performing simple optimizations).

6.21.1. Executing OPT\_EXPR pass (perform const folding).

Optimizing module dadda4.

6.21.2. Executing OPT\_MERGE pass (detect identical cells).

Finding identical cells in module `dadda4'.

Removed a total of 0 cells.

6.21.3. Executing OPT\_DFF pass (perform DFF optimizations).

6.21.4. Executing OPT\_CLEAN pass (remove unused cells and wires).

Finding unused cells or wires in module `dadda4'.

6.21.5. Finished fast OPT passes.



(What is the ABC command?)

ABC is a highly advanced, open-source logic synthesis tool developed by UC Berkeley. Yosys handles the high-level RTL manipulation, but it hands the design over to ABC to perform the heavy-lifting of boolean logic optimization. ABC takes generic logic formulas and maps them down to an optimal arrangement of minimal gates (AND, OR, NOT, etc.) based on advanced algorithms like DAG (Directed Acyclic Graph) covering.

6.22. **Executing ABC pass** (technology mapping using ABC).

6.22.1. Extracting gate netlist of module `dadda4' to `/input.blif'..  
Extracted 80 gates and 88 wires to a netlist network with 8 inputs and 8 outputs.

6.22.1.1. Executing ABC.

Running ABC command: "<yosys-exe-dir>/yosys-abc" -s -f <abc-temp-dir>/abc.script 2>&1

ABC: ABC command line: "source <abc-temp-dir>/abc.script".

ABC:

ABC: + read\_blif <abc-temp-dir>/input.blif

ABC: + read\_library <abc-temp-dir>/stdcells.genlib

ABC: Entered genlib library with 13 gates from file "<abc-temp-dir>/stdcells.genlib".

ABC: + strash

ABC: + dretime

ABC: + map

ABC: + write\_blif <abc-temp-dir>/output.blif

#### 6.22.1.2. Re-integrating ABC results.

|              |                   |    |
|--------------|-------------------|----|
| ABC RESULTS: | ORNOT cells:      | 2  |
| ABC RESULTS: | XNOR cells:       | 4  |
| ABC RESULTS: | AND cells:        | 9  |
| ABC RESULTS: | NAND cells:       | 11 |
| ABC RESULTS: | XOR cells:        | 16 |
| ABC RESULTS: | ANDNOT cells:     | 28 |
| ABC RESULTS: | NOR cells:        | 2  |
| ABC RESULTS: | OR cells:         | 8  |
| ABC RESULTS: | internal signals: | 72 |
| ABC RESULTS: | input signals:    | 8  |
| ABC RESULTS: | output signals:   | 8  |

Removing temp directory.

#### 6.23. Executing OPT pass (performing simple optimizations).

##### 6.23.1. Executing OPT\_EXPR pass (perform const folding).

Optimizing module dadda4.

##### 6.23.2. Executing OPT\_MERGE pass (detect identical cells).

Finding identical cells in module `dadda4'.

Removed a total of 0 cells.

##### 6.23.3. Executing OPT\_DFF pass (perform DFF optimizations).

##### 6.23.4. Executing OPT\_CLEAN pass (remove unused cells and wires).

Finding unused cells or wires in module dadda4..

Removed 0 unused cells and 88 unused wires.

<suppressed ~33 debug messages>

##### 6.23.5. Finished fast OPT passes.

#### 6.24. Executing HIERARCHY pass (managing design hierarchy).

##### 6.24.1. Analyzing design hierarchy..

Top module: dadda4

##### 6.24.2. Analyzing design hierarchy..

Top module: dadda4

Removed 0 unused modules.

## 6.25. Printing statistics.

=== dadda4 ===

|                             |    |
|-----------------------------|----|
| Number of wires:            | 76 |
| Number of wire bits:        | 89 |
| Number of public wires:     | 4  |
| Number of public wire bits: | 17 |
| Number of memories:         | 0  |
| Number of memory bits:      | 0  |
| Number of processes:        | 0  |
| Number of cells:            | 80 |
| \$_ANDNOT_                  | 28 |
| \$_AND_                     | 9  |
| \$_NAND_                    | 11 |
| \$_NOR_                     | 2  |
| \$_ORNOT_                   | 2  |
| \$_OR_                      | 8  |
| \$_XNOR_                    | 4  |
| \$_XOR_                     | 16 |

## 6.26. Executing CHECK pass (checking for obvious problems).

Checking module dadda4...

Found and reported 0 problems.

```
yosys> write_verilog dadda4synth.v
```

7. Executing Verilog backend.

7.1. Executing BMUXMAP pass.

7.2. Executing DEMUXMAP pass.  
Dumping module `dadda4'.

```
yosys>
```